

M15 — Topologia de redes

en CubeSats

Como se conectan los subsistemas y como se ejerce el control
De I2C a CSP, de estrella a pub/sub. 6 misiones analizadas + propuesta INTISAT

Equipo INTISAT

Mayo 2026

Prefacio

Este manual responde a una pregunta que no se hace explícitamente en las reuniones pero que define todo lo demás: **cuando dos subsistemas del CubeSat quieren hablarse, ¿cómo lo hacen?**

La respuesta tiene dos capas:

1. **Capa física:** el bus eléctrico — I2C, SPI, CAN, RS-485. Cables, voltajes, baudrates, terminadores.
2. **Capa lógica:** el protocolo encima del bus — comandos, framing, direcciones, errores, retries. Aquí aparecen CSP, CANopen, Software Bus de cFS, ports de F'.

Por qué importa. En el manual M12 (maquinas de estado) estudiamos *como cada subsistema decide que hacer*. En el M13 estudiamos *como otros lo hicieron*. Pero entre la decisión y la acción hay una capa intermedia: **como viajan los comandos por el satélite**. Esa capa es la *topología de red*.

Que vamos a sacar de aca.

1. Un vocabulario claro: capas OSI adaptadas a CubeSats.
2. Los 5-6 buses físicos que verás en cualquier misión.
3. Las 5 topologías clásicas (estrella, bus, anillo, malla, pub/sub) y donde aparecen.
4. La elección concreta de 6 misiones de referencia (FloripaSat, OreSat, AAUSAT, UPSat, NASA cFS, NASA F'), con su diagrama.
5. Una matriz comparativa.
6. Una propuesta concreta para INTISAT, con direcciones I2C, velocidades y protocolo de aplicación.

Como leerlo. Capítulos 1–3 son el marco conceptual. Capítulos 4–9 son una misión cada uno (independientes, puedes saltar al que te interesa). Capítulo 10 es la matriz. Capítulo 11 cierra con la recomendación para INTISAT.

Índice general

1. Topologia, bus y control: tres ideas distintas	1
1.1. La trampa de mezclar las tres palabras	1
1.2. Las capas (modelo OSI adaptado)	1
1.3. Que pasa si solo implementas L1 y L7 (UPSat style)	1
2. Buses fisicos disponibles	3
2.1. I2C	3
2.2. SPI	4
2.3. CAN / CANopen	4
2.4. RS-485	4
2.5. SpaceWire	5
2.6. UART / RS-232 punto a punto	5
3. Las 5 topologias classicas	6
3.1. Estrella (Star)	6
3.2. Bus / Multidrop	6
3.3. Anillo (Ring)	7
3.4. Malla (Mesh)	7
3.5. Distribuida / Pub-Sub logico	7
4. FloripaSat: estrella SPI con OBDH master	8
4.1. El diseno	8
4.2. Por que SPI y no I2C	9
4.3. Control	9
4.4. Lo que no funciona	9
5. OreSat: bus CAN multidrop con CANopen	10
5.1. El diseno	10
5.2. Por que CAN	10
5.3. Control con CANopen	11
5.4. Lo bueno y lo malo	11
6. AAUSAT: estrella I2C + CSP encima	12
6.1. El diseno	12
6.2. Por que CSP fue revolucionario	13
6.3. Control	13
7. UPSat: estrella I2C sin capa de red	14
7.1. El diseno	14
7.2. Por que funciona	14
7.3. Donde se rompe	15

8. NASA cFS: Software Bus pub/sub	16
8.1. El diseno	16
8.2. Por que esto es topologia	17
8.3. Control	17
9. NASA F': topologia compile-time	18
9.1. El diseno	18
9.2. Por que esto es distinto de cFS	18
9.3. Donde gano F'	19
10. Matriz comparativa	20
10.1. Lectura de la matriz	20
10.2. La decision de INTISAT en contexto	21
11. Como se ejerce el control	22
11.1. Los 4 modelos de control	22
11.2. Flujo temporal de un comando	22
12. Propuesta de topologia para INTISAT	24
12.1. Decision: estrella I2C centrada en CM5	24
12.2. Velocidad y eleccion electrica	25
12.3. Capa de aplicacion: paquetes binarios + CRC16	25
12.4. Roadmap	25
12.5. Lo que tenemos que decidir aun	25
A. Glosario	26
B. Referencias y URLs verificables	28
B.1. Estandares	28
B.2. Protocolos open source para CubeSats	28
B.3. Frameworks NASA	28
B.4. Repos de las misiones	28
B.5. Material de lectura recomendado	29

Capítulo 1

Topologia, bus y control: tres ideas distintas

1.1. La trampa de mezclar las tres palabras

En las reuniones se mezclan tres conceptos que son ortogonales:

- **Bus fisico:** el medio electrico. I2C, SPI, CAN, RS-485, UART. Es *hardware*.
- **Topologia:** la *forma* de las conexiones. Estrella, bus multidrop, anillo, malla, distribuida.
- **Control:** *quien decide que hace cada subsistema*. Centralizado (OBC manda), distribuido (cada uno autonomo), pub/sub (eventos sin jefe), compile-time (la topologia se fija al compilar).

Intuicion

La misma topologia puede ejercer control distinto. Por ejemplo: *estrella* se puede operar como “el OBC manda todo” (FloripaSat) o como “el OBC coordina pero cada perif. tiene su FSM” (AAUSAT). *Mismo cableado, control distinto*.

1.2. Las capas (modelo OSI adaptado)

El modelo OSI clasico tiene 7 capas. En un CubeSat, casi siempre implementas 3 o 4 de ellas:
Para cada capa, una decision concreta:

- **L1 (fisico):** voltaje, baudrate, hilos.
- **L2 (enlace):** formato de frames, CRC, ACK.
- **L3 (red):** direcciones de nodo, routing.
- **L4 (transporte):** confiabilidad, retries, fragmentacion.
- **L7 (aplicacion):** telecomandos, telemetria, file-transfer.

1.3. Que pasa si solo implementas L1 y L7 (UPSat style)

Es lo mas comun en CubeSats 1U sencillos. Tienes I2C como L1, mandas bytes directos como L7 y no hay capas intermedias. **Funciona**, pero pierdes:

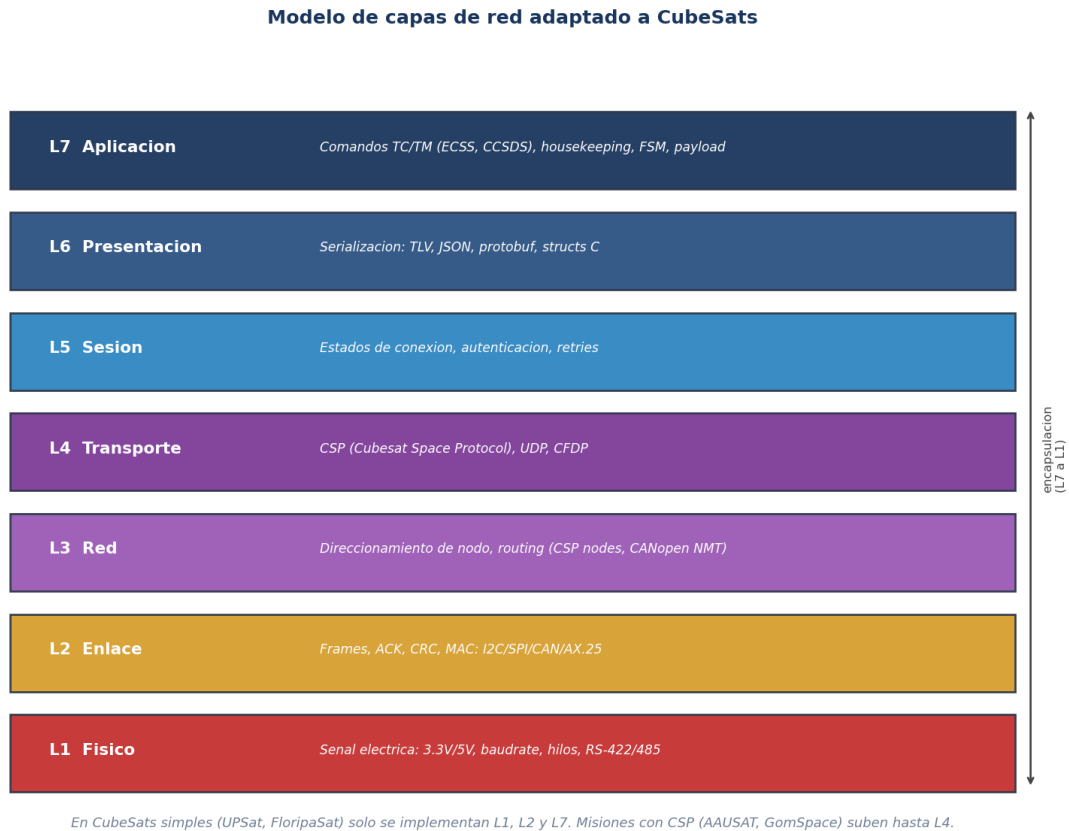


Figura 1.1: Modelo de capas adaptado a CubeSats. CubeSats simples (UPSat, FloripaSat) implementan L1, L2 y L7 directamente. Misiones con CSP suben hasta L4.

- **Retransmision automatica** si una lectura falla.
- **Direccionamiento logico** (todo es “escribi en direccion I2C 0x20”, no “mandale comando GET_TLM al EPS”).
- **Portabilidad:** si manana cambias I2C por CAN, hay que reescribir todo el codigo de aplicacion.

A cambio gana simplicidad. Para una mision de 6 meses sin re-trabajo, suele alcanzar.

Capítulo 2

Buses fisicos disponibles

Antes de hablar de topologias, hace falta entender que opciones electricas existen.

Buses fisicos disponibles en CubeSats: comparacion

Bus	Hilos	Vel max	Topologia	Robustez	Costo	Complej.	Tipico en
I2C	2	0.4 Mbps	multi-drop	**	*	*	UPSat, AAUSAT, INTISAT
SPI	4	50.0 Mbps	estrella (CS)	*	*	**	FloripaSat
UART/RS-232	2	0.1 Mbps	punto a punto	*	*	*	telcomandos legados
RS-485	2	10.0 Mbps	multi-drop	****	**	**	EnduroSat, GomSpace
CAN / CANopen	2	1.0 Mbps	multi-master	*****	**	***	OreSat
Ethernet (Mbps)	4	100.0 Mbps	switched	***	*****	*****	misiones grandes (no CubeSat)
SpaceWire	4	200.0 Mbps	switched	*****	*****	*****	ESA missions (PROBA)

Robustez / Costo / Complejidad: cada * = 1 punto en escala 1-5

Figura 2.1: Buses fisicos tipicos en CubeSats. La columna “Tipico en” indica donde aparece cada bus de verdad.

2.1. I2C

2 hilos (SDA + SCL), open-drain con pull-ups, half-duplex, hasta 400 kHz tipico (1 MHz Fast-mode+ con cuidado).

- **Pro:** simplisimo. Casi todo MCU lo tiene. Direcccionamiento de 7 bits, hasta 127 esclavos teoricos.

- **Contra:** susceptible a glitches (pull-ups marginales, longitud de cable). No hay deteccion de error nativa — si una lectura llega corrompida, no te enteras a menos que agregues CRC a nivel de aplicacion.

Usado en: UPSat, AAUSAT, INTISAT (target), GomSpace (modulos chicos), EnduroSat (algunos modulos).

2.2. SPI

4 hilos (SCLK + MOSI + MISO + CS por esclavo), full-duplex, hasta 50 MHz teorico (1-10 MHz tipico en CubeSat).

- **Pro:** muy rapido. Ideal para sensores de alta tasa (camaras, ADCs).
- **Contra:** necesitas una linea CS por cada esclavo. Si quieres 5 esclavos, son 5 GPIOs adicionales. No hay direccionamiento — el CS *es* la direccion.

Usado en: FloripaSat (bus principal), interfaces SD, sensores rapidos como IMUs.

2.3. CAN / CANopen

2 hilos diferenciales (CAN_H + CAN_L), hasta 1 Mbps en CubeSat (5 Mbps en CAN-FD), multi-master con arbitraje por ID.

- **Pro:** robusto electricamente. Broadcast nativo. Multi-master: cualquier nodo puede iniciar transmision. Arbitraje no-destructivo: en colision gana el ID mas bajo sin perder mensaje.
- **Contra:** requiere transceiver fisico (chip TJA1042 o similar) en cada nodo. Stack de software mas grande que I2C.

CANopen agrega encima un perfil de aplicacion estandar (CiA 301): Object Dictionary, NMT (Network Management), SDOs y PDOs.

Usado en: OreSat (todo el satellite), industria automotriz (de donde sale), GomSpace en sus modulos mas modernos.

2.4. RS-485

2 hilos diferenciales, half-duplex, hasta 10 Mbps a 12 m, o 100 kbps a 1.2 km. Lo mas viejo y mas robusto de la lista.

- **Pro:** inmune a ruido por ser diferencial. Multi-drop sin transceivers complejos.
- **Contra:** half-duplex, sin direccionamiento nativo (lo agregas tu).

Usado en: algunos EPS comerciales (Spacemanic, EnduroSat), interfaces con instrumentos cientificos.

2.5. SpaceWire

4 pares diferenciales, hasta 200+ Mbps. El bus profesional de ESA y NASA para misiones medianas y grandes.

- **Pro:** rapidísimo, switched, ECC. Estandar formal ECSS-E-ST-50-12C.
- **Contra:** sobre-ingeniado para un 1U. Hardware caro (FPGA dedicado).

Usado en: PROBA-1/2/3 (ESA), Mars Reconnaissance Orbiter, JUICE. **No es para CubeSats chicos.**

2.6. UART / RS-232 punto a punto

2 hilos (TX + RX), velocidad libre (9600–921600 baudios típico), solo entre 2 dispositivos.

- **Pro:** 100 % simple. Es lo que usa cualquier radio UHF como interfaz al OBC.
- **Contra:** solo dos puntos. No es un bus, es un cable.

Usado en: interfaz radio-OBC en casi todos los CubeSats, debugging, conexión a PC.

Capítulo 3

Las 5 topologías clásicas

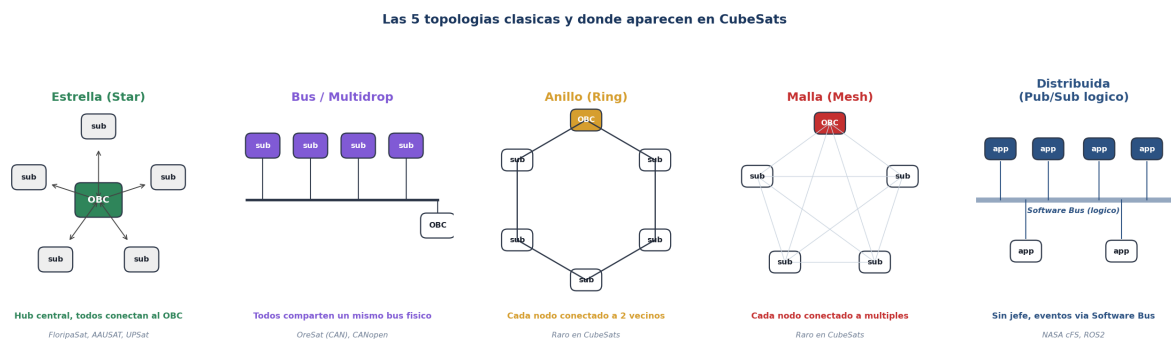


Figura 3.1: Las 5 topologías clásicas y donde aparecen en CubeSats reales. La distribuida *pub/sub* es logica, no física.

3.1. Estrella (Star)

Un nodo central (el OBC) y N periféricos conectados radialmente al centro. **Es la topología mas común en CubeSats.**

Ventajas:

- Simple de razonar: “todos hablan con el OBC”.
- Fácil de debuggear: si X no responde, miras el cable de X.
- Aislamiento de falla parcial: si EPS falla, el ADCS todavía habla con el OBC.

Desventajas:

- El OBC es punto único de fallo. Si se cuelga, nadie habla con nadie.
- Si el bus físico es compartido (I2C), un periférico mal hecho puede colgar el bus para todos.

3.2. Bus / Multidrop

Todos los subsistemas comparten un mismo cable físico (un par diferencial en CAN, dos hilos en I2C). Cualquiera puede enviar; el protocolo decide quien gana.

Ventajas:

- Cableado mínimo: 2-3 hilos para todo el satélite.

- Hot swap real (en CAN): puedes desconectar un nodo y el bus sigue.
- Broadcast nativo: 1 mensaje llega a todos.

Desventajas:

- Mas complejo de debuggear (todos hablan al mismo cable).
- Necesitas un protocolo de arbitraje (CAN lo trae built-in, I2C no).

3.3. Anillo (Ring)

Cada nodo conectado a 2 vecinos. Mensajes circulan hasta llegar al destinatario.

Donde aparece: casi nunca en CubeSats. Mas comun en redes LAN viejas (Token Ring) y en algunas misiones grandes (FireWire networks). Lo nombramos por completitud.

3.4. Malla (Mesh)

Cada nodo conectado a varios. Múltiples caminos hasta cualquier destino.

Donde aparece: raro en CubeSats. Mas comun en redes inalámbricas (Zigbee, Thread). Util cuando la tolerancia a fallos es critica y el costo de cableado no importa.

3.5. Distribuida / Pub-Sub logico

No es una topologia fisica, es logica. El bus puede ser cualquier cosa (I2C, CAN, memoria compartida, ROS topics). La idea es: **nadie es el jefe**. Hay un *Software Bus* al que las aplicaciones publican mensajes y otras se suscriben. Quien publica no sabe quien escucha.

Ventajas:

- Maxima desacoplamiento: agregas/sacas componentes sin tocar a los demas.
- Naturalmente tolerante: si una app cae, las otras siguen.

Desventajas:

- Mas pesado: requiere middleware (cFE, DDS, ROS2).
- Difícil de razonar el orden temporal de eventos.

Donde aparece: NASA cFS Software Bus, ROS2 (en rovers chicos), DDS (Mars 2020).

Capítulo 4

FloripaSat: estrella SPI con OBDH master

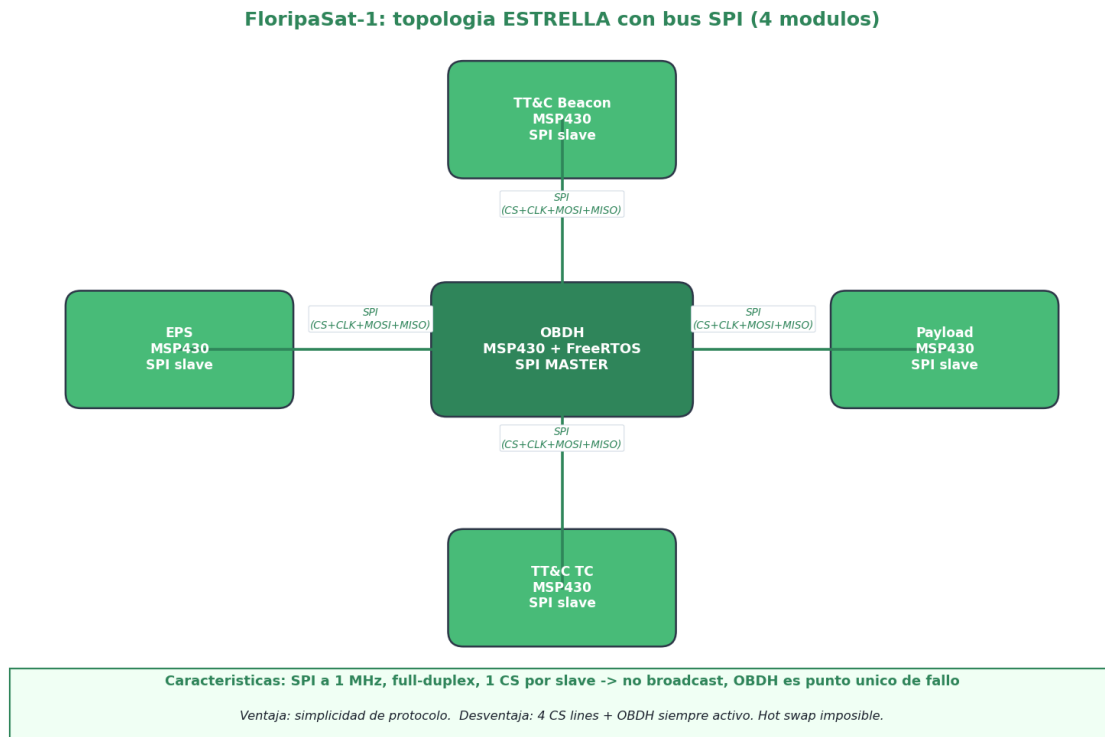


Figura 4.1: Topologia de FloripaSat-1: estrella SPI alrededor del OBDH. Cada perif. es un MSP430 independiente con su firmware.

4.1. El diseno

Cuatro modulos perifericos (EPS, Payload, Beacon TT&C, Telecomando TT&C) conectados al OBDH por SPI. Cada modulo es un MSP430 con FreeRTOS propio.

Señales SPI: CLK, MOSI, MISO compartidas. Una linea CS por slave (4 CS lines en total).

Master: siempre el OBDH. Los esclavos no pueden iniciar transferencias: hay que esperar a que el OBDH les pregunte.

4.2. Por que SPI y no I2C

UFSC eligio SPI por:

1. **Throughput:** SPI a 1 MHz es 2.5x mas rapido que I2C a 400 kHz, y full-duplex.
2. **Inmunidad a fallas en un slave:** en I2C, un slave mal hecho puede colgar el bus entero (clock stretching infinito). En SPI no.
3. **Determinismo:** en SPI el master controla todo el timing. No hay arbitraje.

4.3. Control

100 % centralizado: el OBDH corre la FSM global, lee telemetria de cada slave en un loop y emite comandos cuando hay que cambiar de modo. Los slaves no toman decisiones — solo responden.

4.4. Lo que no funciona

Cuidado

Si el OBDH se cuelga, el satellite muere. No hay forma de que los slaves hablen entre si o con tierra sin pasar por el OBDH. UFSC mitigo esto con un watchdog hardware externo que resetea el OBDH si no se reporta cada N segundos.

Tampoco hay hot swap: si conectas/desconectas un slave en orbita (imposible, pero conceptualmente), el bus SPI se rompe porque las lineas CLK/MOSI quedan flotando.

Capítulo 5

OreSat: bus CAN multidrop con CANopen

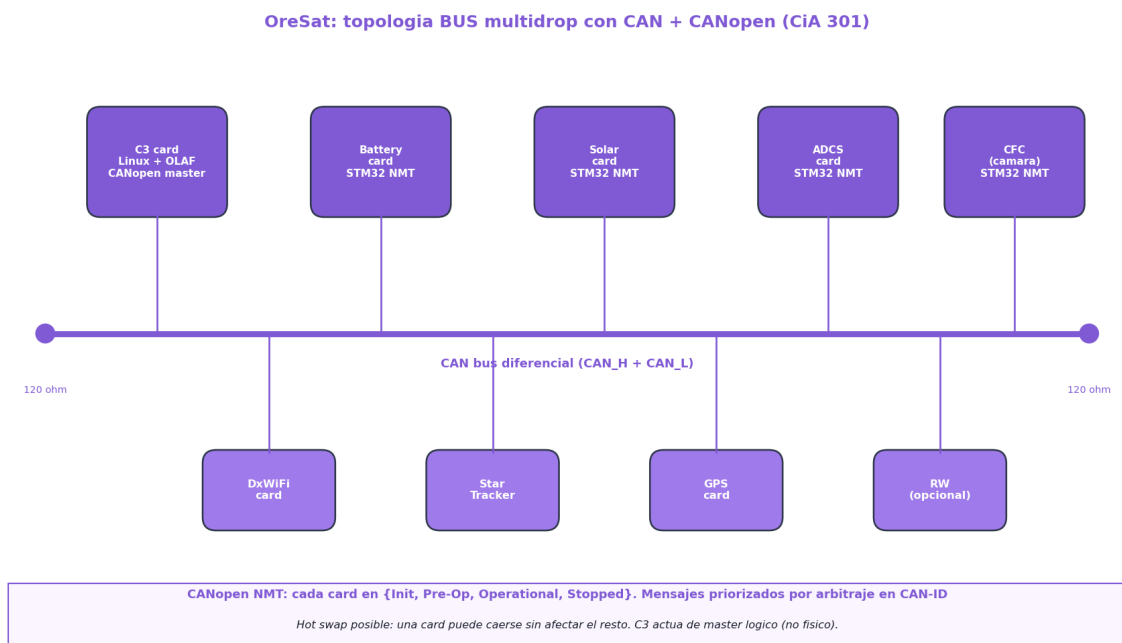


Figura 5.1: OreSat: bus CAN linear con terminadores de 120 ohm y cards CANopen. La C3 actua como master logico, no como master fisico.

5.1. El diseno

Un solo par diferencial CAN_H + CAN_L recorre el cubesat de punta a punta, con terminadores de 120 ohm en cada extremo. Todas las *cards* (subsistemas) cuelgan de este bus.

Cada card es un STM32 con un transceiver CAN (TJA1042 tipico) y un perfil CANopen. La excepcion es la C3 card, que corre Linux con un demonio Python (OLAF) y actua de master logico.

5.2. Por que CAN

OreSat lo eligio por tres razones:

1. **Robustez eléctrica:** señal diferencial, inmune a ruido. CAN nació para autos en condiciones agresivas.
2. **Multi-master:** cualquier card puede transmitir cuando necesita. El arbitraje por ID resuelve colisiones sin perder paquetes.
3. **Estandar abierto:** CANopen CiA 301 es publico. Object Dictionary uniforme entre cards.

5.3. Control con CANopen

Distribuido pero coordinado. Cada card tiene su FSM NMT estandar (Init -> Pre-Op -> Operational -> Stopped). La C3 manda mensajes NMT para forzar transiciones globales (“todos a Pre-Operational”), pero cada card mantiene su propia logica interna.

Detalle tecnico

PDO vs SDO en CANopen.

- **PDO (Process Data Object):** telemetria periodica publicada por la card sin pedido. Cada card publica sus PDOs en sus IDs reservados. Bajo overhead, ideal para housekeeping.
- **SDO (Service Data Object):** lectura/escritura sincrona pedida por otro nodo (tipicamente la C3). Mayor overhead pero confiable.

5.4. Lo bueno y lo malo

Bueno: si una card falla, el bus sigue (terminadores y transceivers la aislan electricamente). Hot swap fisico es teoricamente posible. La C3 puede inicializar el satelite sin saber a priori que cards estan conectadas (NMT *discovery*).

Malo: mas hardware (transceiver en cada card), mas software (stack CANopen), mas masa, mas BOM. Para un 1U universitario es muchas veces over-engineered.

Capítulo 6

AAUSAT: estrella I2C + CSP encima

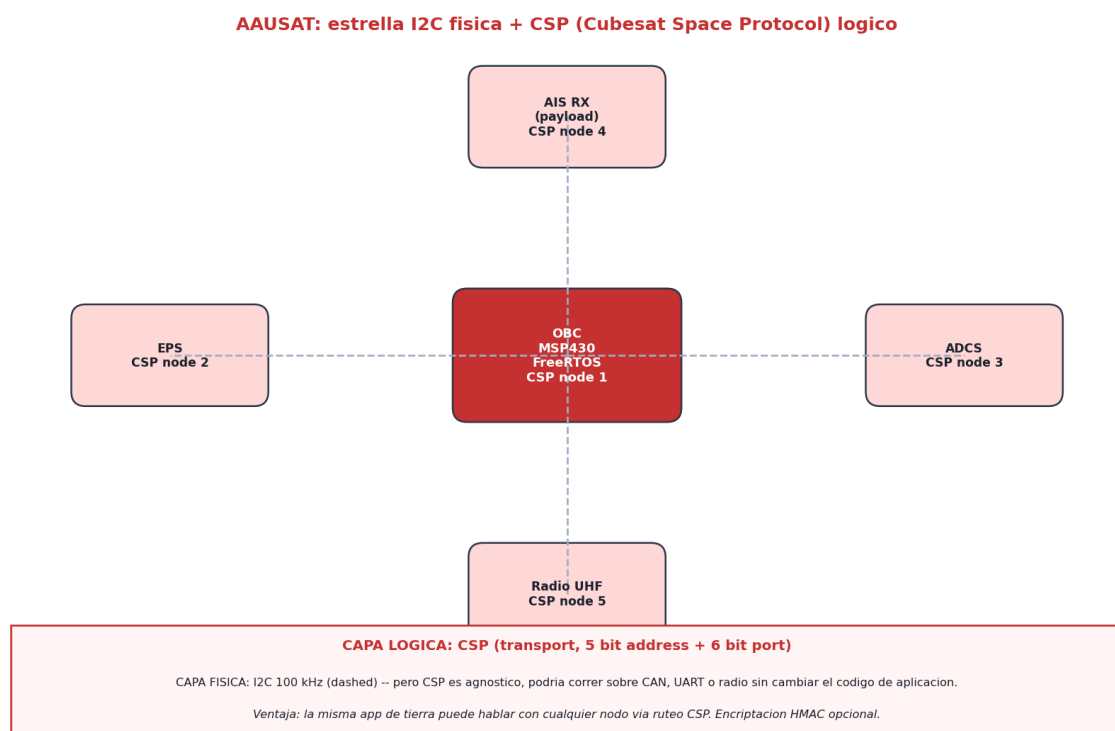


Figura 6.1: AAUSAT: capa fisica estrella I2C (lineas dashed) + capa logica CSP overlay. El codigo de aplicacion habla CSP, no I2C.

6.1. El diseno

Capa fisica: estrella I2C, OBC al centro. Misma que UPSat.

Capa logica: CSP (Cubesat Space Protocol). Es la contribucion grande de Aalborg a la comunidad CubeSat. CSP es un protocolo de red estilo TCP/IP especifico para satelites:

- **Direcciones** de 5 bits (32 nodos posibles).
- **Puertos** de 6 bits (64 servicios por nodo).
- **Header** de 4 bytes con prioridad, source, destination, port y flags (HMAC, RDP, fragmentacion).

- **Capa agnostica del bus:** la misma app corre sobre I2C, CAN, UART o radio sin cambiar el código.

6.2. Por que CSP fue revolucionario

Antes de CSP, cada CubeSat reinventaba su protocolo de comandos. CSP te permite escribir:

```
1 csp_packet_t * packet = csp_buffer_get(100);
2 strcpy((char *)packet->data, "GET_TLM");
3 packet->length = 7;
4
5 /* destino: nodo 2 (EPS), puerto 10 (telemetria) */
6 csp_send(conn, packet, /* timeout */ 1000);
```

Listing 6.1: Envío de comando a otro nodo via CSP

Y eso funciona igual si el EPS esta colgado de I2C, de CAN o llegando por radio. La libreria `libcsp` (open source, MIT) se encarga del ruteo.

6.3. Control

Centralizado en el OBC, pero con una diferencia importante: el OBC no envia bytes a una direccion I2C, envia paquetes a un nodo logico. Si mañana el EPS deja de estar en I2C y pasa a estar accesible por otro bus, el código de aplicación del OBC **no cambia**.

Para INTISAT

INTISAT podría adoptar `libcsp` en v2 sin cambiar la topología física. Quedaría estrella I2C como L1+L2, CSP como L3+L4. Pago: una dependencia C portada a Python (existe wrapper). Gano: rigor de capa de red y futuro-proof.

Capítulo 7

UPSat: estrella I2C sin capa de red

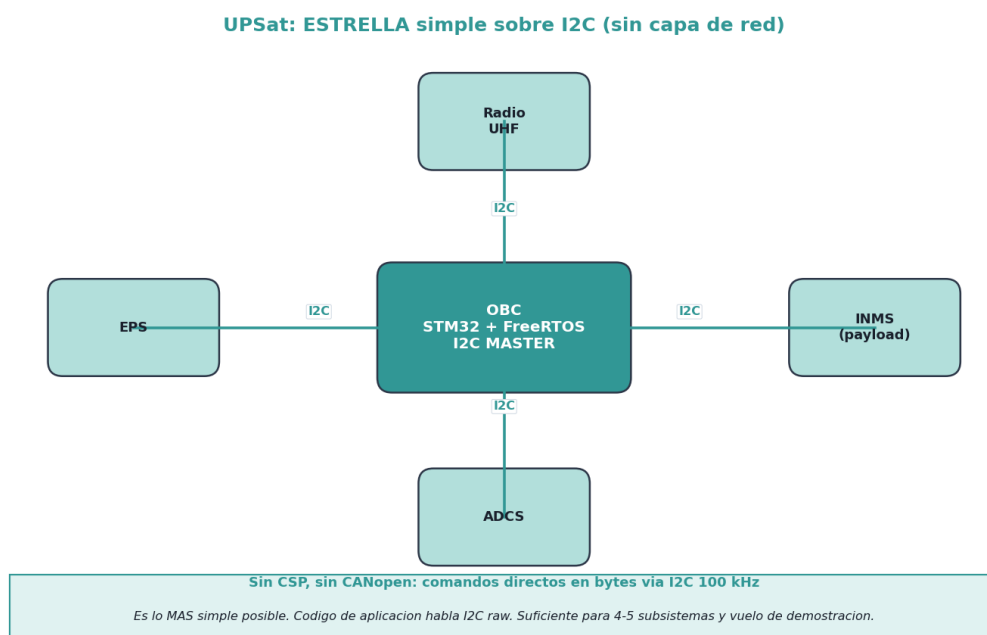


Figura 7.1: UPSat: lo mas simple posible. Estrella I2C, sin CSP, sin CANopen. Comandos directos byte-a-byte.

7.1. El diseno

Estrella I2C, OBC central, 4 perifericos (EPS, INMS payload, radio UHF, ADCS). **Sin capa de red**: el codigo de aplicacion escribe directamente en direcciones I2C.

```
1 uint8_t cmd[4] = { CMD_GET_TLM, 0x00, 0x00, 0x00 };
2 i2c_write(EPS_ADDR, cmd, 4);
3 i2c_read(EPS_ADDR, response_buffer, 16);
4 /* verificar CRC a mano */
```

Listing 7.1: Comando a un periferico en UPSat estilo

7.2. Por que funciona

Para 4 perifericos, una mision de demostracion y un equipo chico, alcanza. El stack de software es 100 lineas de C por periferico. No hay que aprender CSP ni CANopen. El

tiempo de desarrollo es 5x menor que cualquier alternativa con capa de red.

7.3. Donde se rompe

Cuidado

Si la misión crece (más subsistemas, más comandos, más variantes), el código de aplicación empieza a ser un monstruo de `if/else` sobre direcciones I2C. Cualquier cambio en el cableado físico obliga a reescribir el código de comandos.

UPSat duró 6 meses en órbita y fue suficiente. Para una misión de 2 años o con un payload complejo (como INTISAT), probablemente quieras CSP.

Capítulo 8

NASA cFS: Software Bus pub/sub

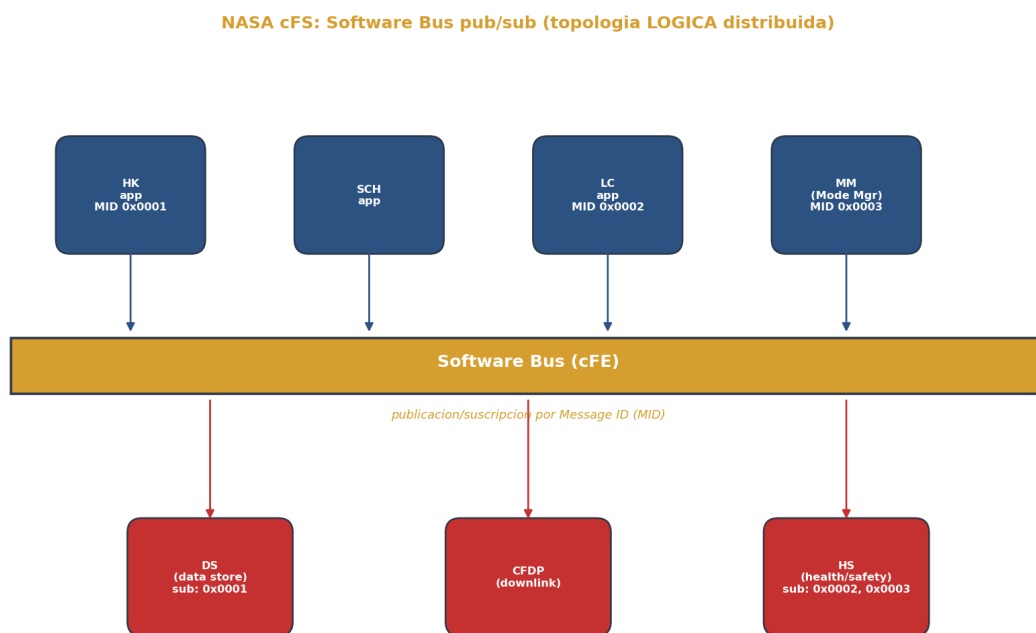


Figura 8.1: NASA cFS: las apps no se conocen entre si. Publican mensajes con un Message ID (MID) y otras apps se suscriben. El Software Bus es el central.

8.1. El diseno

No hay bus fisico definido por cFS. Lo que define cFS es la capa de software: un *Software Bus* corriendo en el OBC al que las apps publican mensajes por *Message ID* (MID, 16 bits) y otras apps se suscriben.

Las apps no se conocen entre si. La app HK publica un mensaje de housekeeping. Las apps DS (Data Storage), CFDP (downlink), HS (Health & Safety) pueden suscribirse, cada una por sus razones, sin que HK sepa quien escucha.

8.2. Por que esto es topologia

Aunque fisicamente todo corre en el mismo procesador, **la topologia logica es distribuida**. Es la misma idea que ROS2 o que DDS: separar el “que publico” del “quien lee”.

Cuando hay multiples procesadores (Lunar IceCube, MMS), el Software Bus se extiende sobre la red real (SpaceWire tipico). Pero el codigo de las apps no se entera.

8.3. Control

No hay un control central, hay reglas. La app **Mode Manager** publica el modo actual. Otras apps *reaccionan* (DS guarda menos datos en modo SAFE, etc).

Intuicion

La diferencia conceptual con FloripaSat o UPSat es enorme: en cFS, nadie “manda”. El estado del satellite es un *shared truth* publicado en el Software Bus. Esto reduce muchisimo el acoplamiento pero requiere disciplina para que todas las apps sepan exactamente en que MIDs publicar.

Capítulo 9

NASA F': topologia compile-time

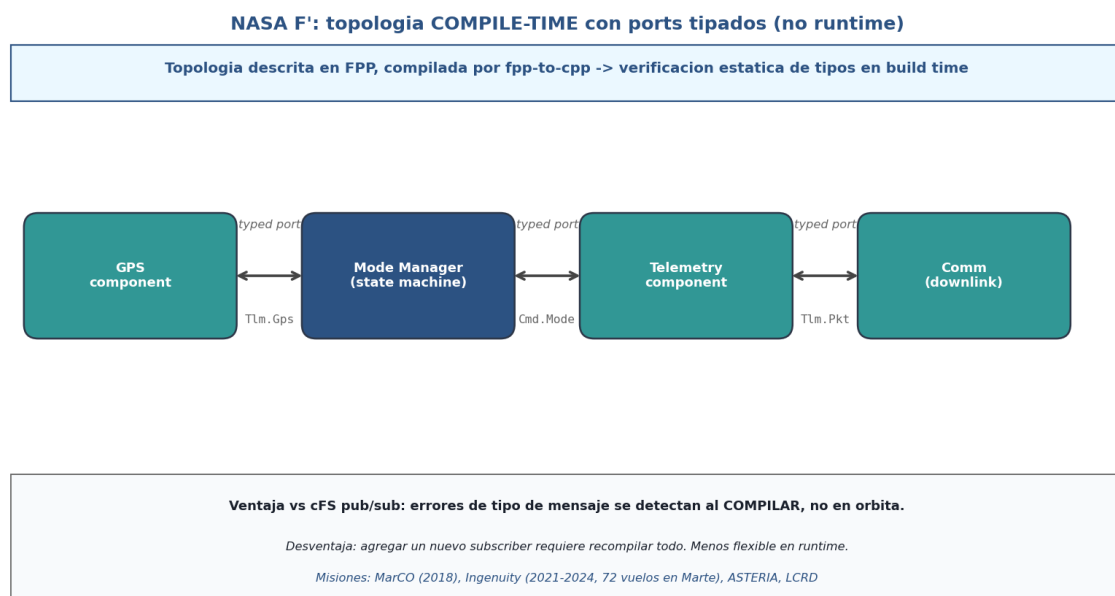


Figura 9.1: F': la topologia se declara en FPP y se compila. No hay “descubrimiento” en runtime, todas las conexiones son estaticas y tipadas.

9.1. El diseno

Componentes conectados por ports tipados. La topologia completa esta en un archivo .fpp (F Prime Prime). Cuando ejecutas `fpp-to-cpp`, el compilador genera el codigo de las conexiones, verifica que los tipos coincidan, y produce C++.

9.2. Por que esto es distinto de cFS

- **cFS (pub/sub):** agregar un suscriptor nuevo es trivial, se hace en runtime. Pero si publicas un MID que nadie escucha, nadie te avisa.
- **F' (compile-time):** si conectas dos ports incompatibles, no compila. Pero agregar un componente nuevo requiere recompilar todo.

Trade-off claro: flexibilidad runtime (cFS) vs verificacion estatica (F').

9.3. Donde gano F'

F' volo en MarCO (2018, primeros CubeSats a Marte), en Ingenuity (72 vuelos en Marte 2021-2024), en ASTERIA, en NEA Scout. Es el framework que mas saco la NASA al espacio profundo en los ultimos 8 anos.

El argumento de JPL fue: *a 5–20 minutos de retardo de senal, queremos errores de tipo detectados en compilacion, no en orbita.* Tiene sentido.

Capítulo 10

Matriz comparativa

Matriz comparativa de topologías y control

	Topología	Bus físico	Capa red	Control	Hot swap?	Falla aislada?
FloripaSat	Estrella	SPI	(ninguna)	Centralizado	No	Parcial
OreSat	Bus multidrop	CAN 1 Mbps	CANopen	Centralizado	Si	Si
AAUSAT	Estrella	I2C	CSP	Centralizado	No	Si (CSP)
UPSat	Estrella	I2C	(ninguna)	Centralizado	No	No
NASA cFS	Pub/Sub logico	depende	SB (cFE)	Distribuido	Si	Si
NASA F ¹	Ports tipados	depende	FPP	Compile-time	No	Si
INTISAT (target)	Estrella	I2C 400 kHz	(CSP opcional)	Centralizado	Limitado	Parcial

INTISAT replica el patron mas simple (estrella + I2C) pero deja la puerta abierta a CSP en v2.

Figura 10.1: Matriz: las 6 misiones de referencia + INTISAT contra 6 dimensiones (topología, bus físico, capa red, control, hot swap, aislamiento de falla).

10.1. Lectura de la matriz

Patrones que valen la pena marcar:

1. **Estrella + I2C** es el patron por defecto en CubeSats universitarios chicos. UPSat, AAUSAT, INTISAT. Lo mas simple que funciona.
2. **Bus multidrop + CAN** aparece cuando hay mas de 5–6 subsistemas y quieres independencia. OreSat. GomSpace en sus modulos modernos.
3. **Capa de red explicita** (CSP, CANopen, SB) es lo que diferencia un proyecto serio de uno de demostracion. UPSat no la tiene y se nota: cambiar cualquier cosa rompe todo.

4. **Pub/sub** (cFS) y **compile-time** (F') son aproximaciones *profesionales*. Requieren equipos con experiencia.

10.2. La decision de INTISAT en contexto

INTISAT replica el patron mas simple: estrella + I2C, sin capa de red en la primera version. Es una decision *deliberada*:

- Sabemos que es lo mas conservador.
- Sabemos que pierde futuro-proof.
- Pero el equipo tiene experiencia con I2C y *cero* experiencia con CSP/CANopen.

La trampa a evitar es decir “mejor ya empezamos con CAN + CANopen porque es mas profesional”. Eso aumenta el riesgo de cronograma sin beneficio claro para una mision de demostracion 1U.

Capítulo 11

Como se ejerce el control

11.1. Los 4 modelos de control

1. **Centralizado puro.** OBC manda comandos, los perifericos obedecen. FloripaSat, UPSat. Simple, riesgo de single-point-of-failure.
2. **Centralizado con autonomia local.** El OBC manda en grande, cada periferico tiene su FSM interna para emergencias. AAUSAT con CSP. INTISAT propuesto.
3. **Distribuido coordinado.** Cada subsistema tiene su propia FSM pero todos siguen un *shared truth*. OreSat con NMT, NASA cFS con Mode Manager.
4. **Compile-time.** La topologia y las conexiones se fijan al compilar. No hay descubrimiento. F'.

11.2. Flujo temporal de un comando

Para hacer concreto el control, veamos como viaja un telecomando desde tierra hasta el subsistema que tiene que ejecutarlo.

Los pasos:

1. **Tierra** envia un paquete TC modulado en UHF.
2. **Radio** demodula, decodifica el frame AX.25.
3. **OBC** valida el comando (CRC, autenticacion) y decide a que subsistema redirigirlo.
4. **Bus I2C:** el OBC escribe el comando en la direccion del EPS.
5. **EPS** ejecuta. Suele ser inmediato (ms).
6. **Respuesta I2C:** el EPS escribe un ACK + status.
7. **Telemetria asincrona:** el OBC compone un paquete TM y lo encola.
8. **Beacon UHF** retransmite la TM en el siguiente ciclo.

Tiempo total tipico: 50–200 ms desde TC valido hasta accion en HW. La respuesta de TM puede ser inmediata o esperar al proximo beacon (cada N segundos).

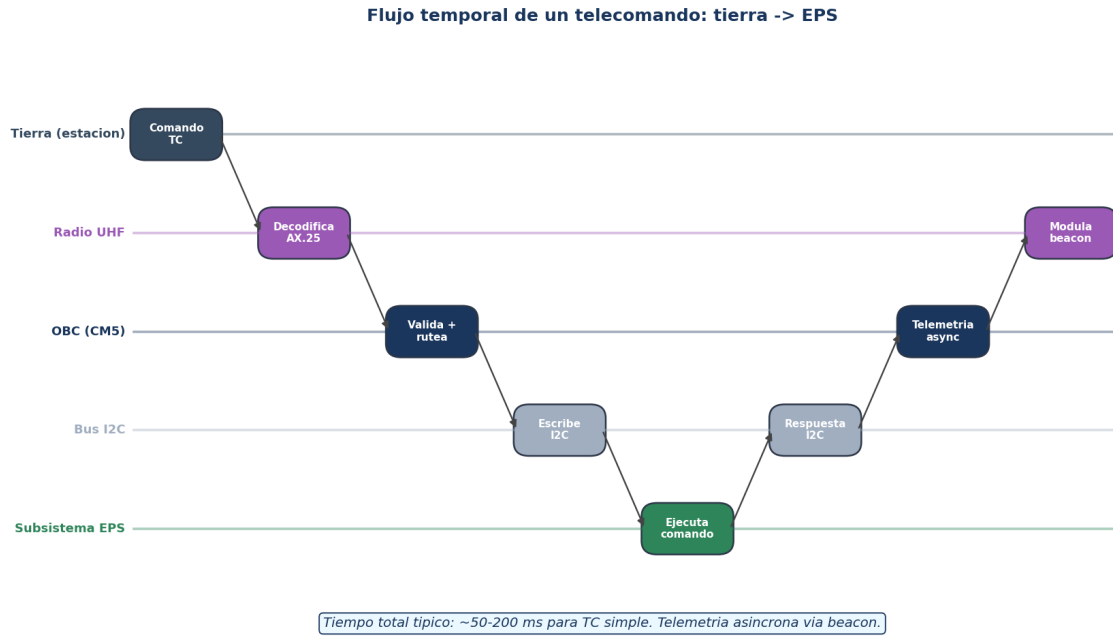


Figura 11.1: Flujo temporal: TC desde tierra hasta ejecución en EPS, con respuesta asincrónica vía beacon. Tiempo total típico 50–200 ms.

Detalle técnico

Por que la TM es asincrónica. Si el TC dispara una acción larga (captura FPM = 180 s), no podemos bloquear el beacon esperando. El OBC encola la TM y la manda cuando hay tiempo de aire. Esto se aleja del modelo request-response sincrónico de HTTP, es más parecido a un sistema de eventos.

Capítulo 12

Propuesta de topologia para INTISAT

12.1. Decision: estrella I2C centrada en CM5

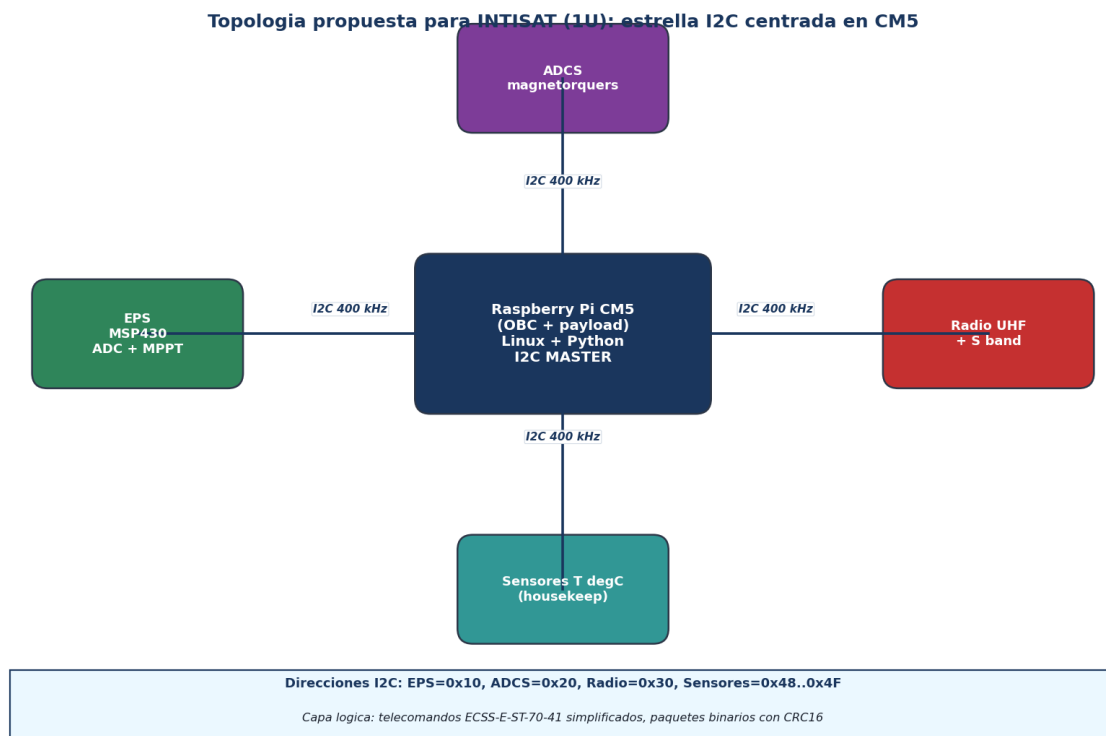


Figura 12.1: Topologia propuesta para INTISAT 1U: CM5 al centro como I2C master, 4 subsistemas perifericos.

Componentes:

- **CM5 (master):** Raspberry Pi Compute Module 5, Linux + Python. OBC + payload integrados en el mismo SoC.
- **EPS:** MSP430 o STM32, en direccion I2C 0x10. Reporta SOC, voltajes de baterias, corrientes de paneles, temperatura.
- **ADCS:** STM32 con magnetorquers, en 0x20. Reporta orientacion estimada por magnetometro.

- **Radio UHF + S:** modulo comercial (EnduroSat o similar), en 0x30. Telemetria/telecomando + downlink payload.
- **Sensores T degC:** TMP117 en cadena, en 0x48..0x4F.

12.2. Velocidad y eleccion electrica

I2C a 400 kHz (Fast-mode). Suficiente para todos los comandos previstos y los chunks de telemetria (<1 kB por comando). No vamos a 1 MHz Fast-mode+ por riesgo de glitches con cables de ~15 cm dentro del 1U.

Pull-ups: 4.7 kohm en SDA y SCL. Estandar Fast-mode con varios dispositivos.

Logica 3.3 V en todo el bus. CM5 ya es 3.3V. Hay que verificar que el EPS lo sea (si es de 5V, level-shifter bidireccional).

12.3. Capa de aplicacion: paquetes binarios + CRC16

Sin CSP en v1. Comandos como:

```

1 # Header: [CMD_ID:1][PAYLOAD_LEN:1][PAYLOAD:N][CRC16:2]
2 def build_tc(cmd_id, payload):
3     pkt = bytes([cmd_id, len(payload)]) + payload
4     crc = crc16_ccitt(pkt)
5     return pkt + crc.to_bytes(2, 'big')
6
7 def send_tc(eps_addr, cmd_id, payload=b''):
8     pkt = build_tc(cmd_id, payload)
9     with smbus2.SMBus(1) as bus:
10         bus.write_i2c_block_data(eps_addr, 0, list(pkt))

```

Listing 12.1: Estructura de telecomando hacia EPS

12.4. Roadmap

Para INTISAT

v1 (mision de demostracion): estrella I2C + paquetes binarios con CRC16. Sin CSP, sin CANopen. Lo minimo que vuela.

v2 (si hay segunda iteracion): agregar libcsp Python sobre el mismo I2C. Ganamos: direccionamiento logico, fragmentacion, retries automaticos. Costo: ~2 semanas de desarrollo + tests.

Lo que NO vamos a hacer: pasar a CAN. Para 4 perifericos no se justifica el costo de transceivers + stack CANopen, sobre todo sin experiencia previa del equipo.

12.5. Lo que tenemos que decidir aun

1. **Direcciones I2C definitivas** por subsistema (propuesto arriba pero falta confirmar con EPS comercial elegido).
2. **Formato exacto del header de paquete** (32 bits? 64 bits? compatible con ECSS?)
3. **Politica de retries:** cuantos antes de declarar error?
4. **Diagnostico de bus:** tenemos que poder leer SCL y SDA desde la CM5 (analizador logico via GPIO) para debugging?

Apéndice A

Glosario

- ACK** Acknowledge, paquete de confirmacion.
- AX.25** Protocolo de capa enlace usado en radio amateur.
- CAN** Controller Area Network. Bus serial diferencial.
- CANopen**
Perfil de aplicacion estandar sobre CAN (CiA 301).
- CCSDS**
Consultative Committee for Space Data Systems.
- cFE** Core Flight Executive. Runtime de NASA cFS.
- CRC** Cyclic Redundancy Check.Codigo de deteccion de errores.
- CS** Chip Select. Linea de seleccion de esclavo en SPI.
- CSP** Cubesat Space Protocol. Capa de red para CubeSats.
- DDS** Data Distribution Service. Middleware pub/sub.
- ECSS** European Cooperation for Space Standardization.
- FPP** F Prime Prime. DSL textual de NASA F'.
- Frame** Unidad de datos en capa enlace.
- Hot swap**
Conectar/desconectar dispositivos sin apagar.
- I2C** Inter-Integrated Circuit. Bus serial 2-hilos.
- MAC** Medium Access Control. Subcapa de enlace.
- MID** Message ID. Identificador de mensaje en cFS Software Bus.
- Multidrop**
Topologia donde varios nodos comparten el mismo bus fisico.
- NMT** Network Management. Servicio CANopen.
- OBC** On-Board Computer.
- OBDH** On-Board Data Handling. Equivalente OBC.

- OSI** Open Systems Interconnection. Modelo de capas de red.
- PDO** Process Data Object. Telemetria periodica en CANopen.
- RS-232 / RS-485**
Estandares electricos serial.
- SB** Software Bus de cFS.
- SDO** Service Data Object. Lectura sincrona en CANopen.
- SOC** State of Charge (bateria).
- SpaceWire**
Estandar ECSS-E-ST-50-12C. Bus alta velocidad para misiones grandes.
- SPI** Serial Peripheral Interface. Bus serial 4-hilos.
- Star / Estrella**
Topologia con hub central.
- TC / TM**
Telecomando / Telemetria.
- Transceiver**
Chip que convierte logica a senal diferencial.
- UART** Universal Asynchronous Receiver Transmitter.

Apéndice B

Referencias y URLs verificables

B.1. Estándares

- I2C spec (NXP / Philips): <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>
- CAN spec (Bosch): <https://www.bosch-semiconductors.com/>
- CANopen CiA 301: <https://www.can-cia.org/can-knowledge/canopen/>
- SpaceWire ECSS-E-ST-50-12C: <https://ecss.nl/>
- AX.25 (radio amateur): TAPR documentacion publica.
- CCSDS Packet Telemetry: <https://public.ccsds.org/Publications/BlueBooks.aspx>

B.2. Protocolos open source para CubeSats

- libcsp (AAUSAT origen): <https://github.com/libcsp/libcsp>
- python-csp: implementaciones varias en Python.
- CANopenNode: <https://github.com/CANopenNode/CANopenNode>

B.3. Frameworks NASA

- NASA cFS: <https://github.com/nasa/cFS>
- NASA F': <https://github.com/nasa/fprime>
- FPP language: <https://nasa.github.io/fpp/>

B.4. Repos de las misiones

- FloripaSat (UFSC): <https://github.com/floripasat>
- OreSat (PSU): <https://github.com/oresat>
- UPSat (Libre Space): <https://gitlab.com/librespacefoundation/upsat/>
- AAUSAT: <https://www.space.aau.dk/cubesat/>

B.5. Material de lectura recomendado

- NASA cFS tutorial (Goddard, YouTube): introduce todo el stack en ~2 horas.
- NASA F' Hello World: <https://nasa.github.io/fprime/Tutorials/HelloWorld/Tutorial.html>
- “CubeSat Communication System, a New Open Source Design” — Aalborg paper sobre CSP.